Appendix A

The standard library atlas

know the whole map, learn the roads you use

Python ships 195 public modules. Nobody knows them all, and nobody should: the skill is knowing **which one exists for a job** and learning it on demand (Ch 7). This map is generated from `sys.stdlib_module_names`, so it is complete for the Python that built this book; the build fails if a module is missing.

tier 1: use it weekly (48) **tier 2: know it exists (68)** **tier 3: map only (79)**

Text
string **re** **difflib** **textwrap** **unicodedata** **stringprep** **readline** **rlcompleter**

Functional tools
itertools **functools** **operator**

Compression and archives
compression **zlib** **gzip** **bz2** **lzma** **zipfile** **tarfile**

Operating system
os **io** **time** **logging** **platform** **errno** **ctypes** **argparse** **optparse** **getopt** **getpass** **shlex** **curses**

Internet data
email **json** **mailbox** **mimetypes** **base64** **binascii** **quopri**

Multimedia
wave **colorsys**

Graphical interfaces
tkinter **turtle** **turtledemo** **idlelib**

Python runtime
sys **sysconfig** **builtins** **__future__** **warnings** **dataclasses** **contextlib** **abc** **atexit** **traceback** **gc** **inspect** **site** **this** **antigravity**

Unix only
posix **pwd** **grp** **termios** **tty** **pty** **fcntl** **resource** **syslog**

Binary data
struct **codecs** **encodings**

Numbers and maths
numbers **math** **cmath** **decimal** **fractions** **random** **statistics**

Persistence
pickle **copyreg** **shelve** **marshal** **dbm** **sqlite3**

File formats
csv **configparser** **tomllib** **netrc** **plistlib**

Networking and signals
socket **ssl** **select** **selectors** **signal** **mmap**

Markup
html **xml** **pyexpat**

Internationalisation
gettext **locale**

Debugging and profiling
bdb **faulthandler** **pdb** **timeit** **trace** **tracemalloc** **cProfile** **profile** **pstats**

Language services
ast **symtable** **token** **keyword** **tokenize** **tabnanny** **pyclbr** **py_compile** **compileall** **dis** **pickletools** **opcode** **sre_compile** **sre_constants** **sre_parse**

Data types
datetime **zoneinfo** **calendar** **collections** **heapq** **bisect** **array** **weakref** **types** **copy** **pprint** **reprlib** **enum** **graphlib**

Files and directories
pathlib **stat** **filecmp** **tempfile** **glob** **fnmatch** **linecache** **shutil** **fileinput** **genericpath** **posixpath** **ntpath**

Cryptographic services
hashlib **hmac** **secrets**

Concurrency
threading **multiprocessing** **concurrent** **subprocess** **sched** **queue** **contextvars** **asyncio**

Internet protocols
webbrowser **wsgiref** **urllib** **http** **ftplib** **poplib** **imaplib** **smtplib** **uuid** **socketserver** **xmlrpc** **ipaddress**

Development tools
typing **annotationlib** **pydoc** **pydoc_data** **doctest** **unittest** **venv** **ensurepip** **zipapp**

Interpreters and imports
code **codeop** **cmd** **zipimport** **pkgutil** **modulefinder** **runpy** **importlib**

Windows only
msvcrt **winreg** **winsound** **nt** **nturl2path**

How to read the lists

Each job group lists its modules with a tier dot and the chapters of this book that use them (found automatically from the snippets). Tier 1 is the working set of a Python programmer: learn its API until you stop looking it up. Tier 2: know what it's for, so you reach for it instead of writing it yourself. Tier 3: recognise the name.

```python
import sys

names = sorted(sys.stdlib_module_names)
public = [n for n in names if not n.startswith("_") or n == "__future__"]
print(sys.version.split()[0], len(names), "names,", len(public), "public")
print([n for n in public if n.startswith("con")])
```

```
3.14.6 297 names, 195 public
['concurrent', 'configparser', 'contextlib', 'contextvars']
```

RULE OF THUMB

A leading underscore means private. `_json` is the C speed-up that `json` uses when it can, `_pydecimal` the pure-Python fallback for `decimal`, `_thread` the layer under `threading`. Import the public module; its internals can change in any release. The grey box lists them only so the map is complete.

Underscore modules: private internals and C accelerators (102)

`_abc` `_aix_support` `_android_support` `_apple_support` `_ast` `_ast_unparse` `_asyncio` `_bisect` `_blake2` `_bz2` `_codecs` `_codecs_cn` `_codecs_hk` `_codecs_iso2022` `_codecs_jp` `_codecs_kr` `_codecs_tw` `_collections` `_collections_abc` `_colorize` `_compat_pickle` `_contextvars` `_csv` `_ctypes` `_curses` `_curses_panel` `_datetime` `_dbm` `_decimal` `_elementtree` `_frozen_importlib` `_frozen_importlib_external` `_functools` `_gdbm` `_hashlib` `_heapq` `_hmac` `_imp` `_interpchannels` `_interpqueues` `_interpreters` `_io` `_ios_support` `_json` `_locale` `_lsprof` `_lzma` `_markupbase` `_md5` `_multibytecodec` `_multiprocessing` `_opcode` `_opcode_metadata` `_operator` `_osx_support` `_overlapped` `_pickle` `_posixshmem` `_posixsubprocess` `_py_abc` `_py_warnings` `_pydatetime` `_pydecimal` `_pyio` `_pylong` `_pyrepl` `_queue` `_random` `_remote_debugging` `_scproxy` `_sha1` `_sha2` `_sha3` `_signal` `_sitebuiltins` `_socket` `_sqlite3` `_sre` `_ssl` `_stat` `_statistics` `_string` `_strptime` `_struct` `_suggestions` `_symtable` `_sysconfig` `_thread` `_threading_local` `_tkinter` `_tokenize` `_tracemalloc` `_types` `_typing` `_uuid` `_warnings` `_weakref` `_weakrefset` `_winapi` `_wmi` `_zoneinfo` `_zstd`

IN OTHER LANGUAGES

Every language has an atlas like this: the C standard library (small: `stdio.h`, `stdlib.h`, `string.h`...), Java's `java.base` packages, Rust's `std` modules. Python's is unusually large ("batteries included"), which is why Ch 19's question 'do I need a package at all?' so often ends with no.

Text		chapters
`string`	constants (`ascii_letters`, `digits`), `Template` substitution	9 · 13
`re`	regular expressions: search, match, split, substitute	9
`difflib`	diffs between sequences; close-match suggestions	7 · 9 · 19
`textwrap`	wrap, fill, indent and dedent paragraphs	9 · 14
`unicodedata`	character names, categories, normalisation (NFC/NFKC)	9
`stringprep`	internet string preparation tables (RFC 3454)	
`readline`	line editing and history for `input()` on Unix	
`rlcompleter`	tab completion of Python names for readline	

Binary data

		chapters
● `struct`	pack and unpack C structs and binary file headers	8 · 15
● `codecs`	the encoder/decoder registry behind `str.encode`	
● `encodings`	the codec implementations (utf_8, latin_1, idna...)	

Data types

		chapters
● **`datetime`**	dates, times, durations; aware vs naive	8 · 11 · 17
● **`zoneinfo`**	IANA time zones for aware datetimes	11
● `calendar`	calendars, leap years, weekday arithmetic	
● **`collections`**	`deque`, `Counter`, `defaultdict`, `namedtuple`, `ChainMap`	1 · 4 · 10 ...
● **`heapq`**	a min-heap on a plain list; priority queues, top-k	10 · 20 · 21
● **`bisect`**	binary search and insertion into sorted lists	10 · 20
● `array`	compact typed arrays of numbers	3
● `weakref`	references that don't keep an object alive; caches	
● `types`	names for built-in types, `SimpleNamespace`, `MappingProxyType`	
● **`copy`**	shallow and deep copies	3
● `pprint`	pretty-print nested data structures	
● `reprlib`	size-limited `repr()` for huge or recursive objects	
● **`enum`**	named constants: `Enum`, `IntEnum`, `Flag`, `StrEnum`	5
● `graphlib`	`TopologicalSorter`: dependency order and cycle detection	21

Numbers and maths

		chapters
● `numbers`	abstract number classes (`Integral`, `Real`...)	
● **`math`**	floats and integers: `sqrt`, `isclose`, `comb`, `gcd`, `fsum`	1 · 7 · 11 ...
● `cmath`	the same functions for complex numbers	
● `decimal`	exact decimal arithmetic with chosen rounding (money)	5 · 11
● `fractions`	exact rational numbers	11
● **`random`**	pseudo-random numbers for simulations (never secrets)	0 · 2 · 11 ...
● **`statistics`**	mean, median, stdev, quantiles, correlation	7 · 11 · 18

Functional tools

		chapters
● **`itertools`**	lazy iterator building blocks: chain, groupby, product...	10
● **`functools`**	`cache`, `partial`, `reduce`, `wraps`, `total_ordering`	4 · 5 · 10 ...
● **`operator`**	operators as functions: `itemgetter`, `attrgetter`	4 · 10

Files and directories

		chapters
● **`pathlib`**	paths as objects: join, glob, read, write	0 · 2 · 8 ...
● `stat`	decode `os.stat()` mode bits	
● `filecmp`	compare files and directory trees	
● **`tempfile`**	temporary files and directories, cleaned up for you	6 · 8 · 17
● `glob`	shell-style wildcard file matching	8
● `fnmatch`	match one name against a wildcard pattern	
● `linecache`	random access to lines of source files (tracebacks)	
● **`shutil`**	copy, move, delete trees; archives; `which`	8
● `fileinput`	loop over lines of many files or stdin	
● `genericpath`	shared code behind `os.path` (internal)	
● `posixpath`	`os.path` on Unix	
● `ntpath`	`os.path` on Windows	

Persistence

		chapters
●	`pickle`	serialise Python objects (never load untrusted data) — 8
●	`copyreg`	register how pickle and copy handle a type
●	`shelve`	a persistent dict of pickled values
●	`marshal`	the internal format of .pyc files
●	`dbm`	simple key-value database files
●	**`sqlite3`**	a full SQL database in one file — 0 · 6 · 7 …

Compression and archives

		chapters
●	`compression`	new home of the codecs, including `compression.zstd` (3.14) — 8
●	`zlib`	deflate compression and CRC-32 — 8
●	`gzip`	read and write .gz files — 8
●	`bz2`	bzip2 compression — 8
●	`lzma`	xz/LZMA compression — 8
●	`zipfile`	read and write ZIP archives — 7 · 8
●	`tarfile`	read and write tar archives (use extraction filters) — 8

File formats

		chapters
●	**`csv`**	comma-separated files, including quoting rules — 8 · 18
●	`configparser`	INI-style configuration files
●	`tomllib`	read TOML (pyproject.toml); read-only — 8 · 17
●	`netrc`	parse ~/.netrc login files
●	`plistlib`	Apple property lists

Cryptographic services

		chapters
●	**`hashlib`**	SHA-2/SHA-3/BLAKE2 digests; `scrypt`, `pbkdf2_hmac` — 7 · 13 · 19
●	**`hmac`**	keyed message authentication; `compare_digest` — 0 · 7 · 13
●	**`secrets`**	unguessable tokens and choices for security — 2 · 11 · 13

Operating system

		chapters
●	**`os`**	processes, environment, files, directories — 6 · 8 · 14 …
●	**`io`**	the stream classes behind `open()`; `StringIO`, `BytesIO` — 2 · 7 · 14 …
●	**`time`**	clocks (`perf_counter`, `monotonic`), sleep, timestamps — 4 · 6 · 10 …
●	**`logging`**	levelled, configurable logging for libraries and apps — 6 · 14
●	`platform`	which OS, machine and Python this is
●	`errno`	symbolic names for OS error codes
●	`ctypes`	call C functions in shared libraries
●	**`argparse`**	command-line parsing with help and subcommands — 7 · 14
●	`optparse`	the older command-line parser
●	`getopt`	C-style command-line parsing
●	`getpass`	read a password without echoing it
●	`shlex`	split and quote shell-style command lines safely — 14
●	`curses`	full-screen terminal interfaces

Concurrency

		chapters
●	**`threading`**	threads, locks, events, conditions — 6 · 15 · 16 …
●	`multiprocessing`	processes with queues, pipes, shared memory, pools — 16
●	**`concurrent`**	`concurrent.futures`: thread and process pools — 16
●	**`subprocess`**	run other programs; pipes and exit codes — 7 · 14
●	`sched`	a simple event scheduler
●	**`queue`**	thread-safe FIFO, LIFO and priority queues — 16

● `contextvars`	context-local state (per task in asyncio)	16
● **`asyncio`**	async/await: event loop, tasks, streams	0 · 7 · 16

Networking and signals

chapters

● **`socket`**	TCP and UDP sockets	15
● `ssl`	TLS for sockets: certificates, contexts	13 · 15
● `select`	wait for I/O on many file descriptors	
● `selectors`	high-level I/O multiplexing (epoll/kqueue)	
● `signal`	handle Unix signals (SIGINT, SIGTERM...)	14
● `mmap`	memory-mapped files	

Internet data

chapters

● `email`	build and parse email messages (MIME)	15
● **`json`**	JSON encode and decode	0 · 2 · 7 ...
● `mailbox`	mbox and Maildir mailboxes	
● `mimetypes`	guess a MIME type from a file name	
● **`base64`**	base64/base32/base85 encodings (not encryption)	7 · 13
● `binascii`	low-level binary-to-ASCII conversions	
● `quopri`	quoted-printable encoding	

Markup

chapters

● `html`	`escape`, `unescape`, and a simple HTML parser	9
● `xml`	ElementTree, DOM, SAX (use defusedxml for untrusted input)	
● `pyexpat`	the Expat parser behind `xml`	

Internet protocols

chapters

● `webbrowser`	open a URL in the user's browser	
● `wsgiref`	reference WSGI server and utilities	
● **`urllib`**	URLs: parse, quote, fetch (`urllib.request`)	7 · 13 · 15 ...
● **`http`**	`http.client`, `http.server`, status codes, cookies	7 · 13 · 15 ...
● `ftplib`	FTP client	
● `poplib`	POP3 client	
● `imaplib`	IMAP4 client	
● `smtplib`	send mail over SMTP	15
● `uuid`	universally unique identifiers (uuid4, uuid7)	13
● `socketserver`	a framework for TCP/UDP servers	15
● `xmlrpc`	XML-RPC client and server	
● `ipaddress`	IPv4/IPv6 addresses and networks	

Multimedia

chapters

● `wave`	read and write WAV files	
● `colorsys`	convert between RGB, HLS, HSV, YIQ	

Internationalisation

chapters

● `gettext`	translations of messages	
● `locale`	locale-aware formatting and sorting	

Graphical interfaces

chapters

● `tkinter`	the Tk GUI toolkit	7
● `turtle`	turtle graphics for teaching	
● `turtledemo`	demos for turtle	

idlelib	the IDLE editor's code	

Development tools

		chapters
typing	type hints: generics, `Protocol`, `TypedDict`...	5 · 10 · 17
annotationlib	inspect annotations, which are lazy since 3.14	
pydoc	documentation from docstrings (`help()`)	
pydoc_data	data files for pydoc	
doctest	run the examples in docstrings as tests	17
unittest	the test framework; `unittest.mock`	0 · 7 · 17 ...
venv	virtual environments	17 · 19
ensurepip	bootstrap pip into an environment	
zipapp	bundle a program into a runnable .pyz	

Debugging and profiling

		chapters
bdb	the debugger framework under pdb	
faulthandler	dump tracebacks on crashes and hangs	
pdb	the interactive debugger (`breakpoint()`)	0 · 2
timeit	time small snippets reliably	0 · 17 · 18 ...
trace	trace execution and line coverage	
tracemalloc	find where memory is allocated	
cProfile	deterministic profiler (C)	0 · 17
profile	the same profiler in pure Python	
pstats	sort and print profiler results	17

Python runtime

		chapters
sys	interpreter state: argv, path, modules, exit, stdout	2 · 3 · 4 ...
sysconfig	install paths and build configuration	19
builtins	the module holding `len`, `print`, `open`...	
__future__	opt in to future language behaviour	
warnings	issue and filter warnings (deprecations)	
dataclasses	classes from field declarations	5 · 8 · 12 ...
contextlib	`contextmanager`, `suppress`, `ExitStack`	6 · 12
abc	abstract base classes	5
atexit	run functions at interpreter exit	
traceback	format and print exceptions and stacks	6
gc	the cycle collector: collect, inspect, disable	3
inspect	look inside live objects: signatures, source, frames	7
site	start-up path configuration (site-packages)	
this	the Zen of Python (`import this`)	
antigravity	an xkcd joke (opens a browser)	

Interpreters and imports

		chapters
code	build your own interactive console	
codeop	compile possibly incomplete input (REPLs)	
cmd	line-oriented command interpreters	
zipimport	import modules from ZIP files	
pkgutil	iterate over and load package data	
modulefinder	find the modules a script uses	
runpy	run modules as scripts (`python -m`)	
importlib	the import system; `importlib.metadata`, `.resources`	2 · 19

Language services

		chapters
●	`ast`	parse source into a tree; analyse or rewrite code 2 · 9
●	`symtable`	the compiler's symbol tables
●	`token`	token type constants
●	`keyword`	is this string a keyword?
●	`tokenize`	split source into tokens (highlighters, linters) 2
●	`tabnanny`	detect ambiguous indentation
●	`pyclbr`	browse classes and functions without importing
●	`py_compile`	compile one file to .pyc
●	`compileall`	compile a whole tree to .pyc
●	`dis`	disassemble bytecode 2
●	`pickletools`	inspect and optimise pickles
●	`opcode`	bytecode opcode tables
●	`sre_compile`	deprecated alias of `re` internals
●	`sre_constants`	deprecated alias of `re` internals
●	`sre_parse`	deprecated alias of `re` internals

Unix only

		chapters
●	`posix`	the raw OS calls behind `os` on Unix
●	`pwd`	the password database (users)
●	`grp`	the group database
●	`termios`	terminal control
●	`tty`	raw and cbreak terminal modes
●	`pty`	pseudo-terminals
●	`fcntl`	file and I/O control, file locks
●	`resource`	process resource limits and usage
●	`syslog`	the Unix system log

Windows only

		chapters
●	`msvcrt`	Windows C runtime: console keys, file locking
●	`winreg`	the Windows registry
●	`winsound`	play sounds on Windows
●	`nt`	the raw OS calls behind `os` on Windows
●	`nturl2path`	file URL to path conversion (internal)